



RAPPORT DE PROJET FIN D'ANNEE

Sujet : FormationPro

*Mise en place d'une plateforme de gestion des formations,
des compétences
et du suivi de progression*

Réalisé par:

Emna Samti

Encadrant académique:

Mr. Mohamed chiheb Ben
Chaabene

Filière: Ingénierie Informatique

Spécialité: Cloud Computing & Virtualisation

Remerciements

C'est avec un grand plaisir que je consacre cette page en signe de gratitude et de profonde reconnaissance à toute personne ayant contribué, de près ou de loin, à l'orientation de mon travail et à la réalisation de ce projet.

Je tiens particulièrement à remercier mon encadrant académique, *Mohamed chiheb Ben Chaabene*, pour ses conseils avisés, sa disponibilité et son accompagnement tout au long de ce projet.

Je remercie également toutes les personnes ayant contribué, directement ou indirectement, à la réussite de ce travail.

Résumé

Ce rapport présente le projet de fin d'études intitulé **FormationPro**, une plateforme web full-stack dédiée à la gestion des formations professionnelles, au suivi des compétences et au pilotage de la progression des employés au sein d'une organisation.

Face à la fragmentation des outils existants (fichiers Excel, e-mails et solutions non centralisées), FormationPro propose une solution unifiée, sécurisée et multi-rôles permettant de digitaliser l'ensemble du cycle de formation : du catalogue à l'inscription, jusqu'à l'évaluation d'impact.

Le système repose sur une architecture moderne basée sur des micro-services logiques et une séparation claire entre frontend et backend. Le backend est développé avec **Spring Boot (Java 17)** et sécurisé via **Spring Security et JWT**, tandis que le frontend utilise **React, TypeScript et Vite**. Les données sont stockées dans **MariaDB** via **Spring Data JPA / Hibernate**.

Sur le plan DevOps et cloud, l'application est entièrement conteneurisée avec **Docker** et déployée sur **Amazon EKS (Kubernetes)**. L'infrastructure est automatisée via **Terraform (Infrastructure as Code)**, avec un pipeline **CI/CD GitHub Actions** assurant le build, les tests, l'analyse de qualité via **SonarCloud**, puis le déploiement sur AWS. Les images Docker sont stockées dans **Amazon ECR** et le trafic est exposé via un **AWS Application Load Balancer (ALB)**.

Le projet suit la méthodologie **Agile Scrum** en quatre sprints, couvrant progressivement l'authentification, la gestion des formations, le pilotage des équipes et les modules d'analyse et de reporting.

Table des matières

Introduction Générale	1
1 Étude des besoins	2
1.1 Présentation du contexte	2
1.1.1 Contexte du projet	2
1.1.2 Problèmes identifiés	2
1.2 Analyse des besoins	3
1.2.1 Besoins fonctionnels	3
1.2.1.1 BF1 Authentification et gestion des comptes	3
1.2.1.2 BF2 Gestion des utilisateurs (RH)	3
1.2.1.3 BF3 Catalogue de formations	3
1.2.1.4 BF4 Inscriptions et progression	3
1.2.1.5 BF5 Compétences (Skill Tree)	3
1.2.1.6 BF6 Objectifs de formation	3
1.2.1.7 BF7 Feedback et évaluation d'impact	3
1.2.1.8 BF8 Notifications	4
1.2.1.9 BF9 Rapports	4
1.2.2 Besoins non fonctionnels	4
1.2.3 Acteurs du système	4
1.3 Technologies et outils	5

1.3.1	Frontend	5
1.3.2	Backend	6
1.3.3	Base de données	6
1.3.4	DevOps et déploiement	7
1.3.5	CI/CD et qualité	7
2	Conception	9
2.1	Méthodologie Agile Scrum	9
2.1.1	Présentation de Scrum	9
2.1.2	Rôles Scrum	9
2.1.3	Cérémonies Scrum	9
2.1.4	Definition of Done (DoD)	9
2.1.5	Gestion des risques.....	10
2.2	Backlog produit et user stories	10
2.3	Organisation des sprints	11
2.3.1	Sprint 1 Socle : Authentification, Utilisateurs et Sécurité.....	12
2.3.2	Sprint 2 Catalogue, Inscriptions et Compétences	13
2.3.3	Sprint 3 Pilotage Manager : Équipe, Objectifs, Feedback et No- tifications.....	13
2.3.4	Sprint 4 Mesure d'impact, Rapports et Déploiement AWS.....	14
2.4	Conception UML.....	14
2.4.1	Diagramme de cas d'utilisation	14
2.4.2	Diagramme de classes.....	15
2.4.3	Diagramme de séquence Authentification (Login).....	16
2.4.4	Diagramme de séquence Le responsable valide la formation des employés.....	16
2.4.5	Diagramme de séquence Le responsable valide la formation des employés.....	17
2.4.6	Diagramme de séquence le RH génère un rapport sur les écarts de compétences	17
2.5	Architecture du système.....	18
2.5.1	Architecture globale	18
2.5.2	Architecture backend en couches.....	18
2.5.3	Architecture cloud AWS EKS	19
2.5.4	Flux de données et sécurité	20
3	Réalisation et Tests	21
3.1	Environnement de développement.....	21
3.1.1	Outils et frameworks.....	21

3.2	Implémentation	21
3.2.1	Module Authentification et Sécurité JWT	21
3.2.2	Module Catalogue de Formations	22
3.2.3	Module Inscriptions et Progression.....	22
3.2.4	Module Compétences (Skill Tree)	23
3.2.5	Module Objectifs et Pilotage Manager.....	23
3.2.6	Module Feedback et Évaluations d'impact.....	24
3.2.7	Module Rapports	25
3.3	Tests.....	26
3.3.1	Stratégie de tests.....	26
3.3.2	Tests fonctionnels	27
3.3.3	Tests de sécurité	27
3.3.4	Tests d'intégration.....	27
3.4	Déploiement	28
3.4.1	Infrastructure AWS (Terraform)	28
3.4.2	Pipeline CI/CD (GitHub Actions)	29
3.4.3	Manifests Kubernetes (EKS).....	29
3.5	Monitoring et Observabilité.....	30
3.5.1	Healthchecks Kubernetes	30
3.5.2	Dashboard CloudWatch	30
3.5.3	Alarmes CloudWatch	32
	Conclusion Générale	33
	Bibliographie	35

Table des figures

2.1	Diagramme de cas d'utilisation FormationPro	15
2.2	Diagramme de classes Domaine métier FormationPro	16
2.3	Diagramme de séquence Flux de connexion JWT	16
2.4	Diagramme de séquence Inscription à une formation.....	16
2.5	Diagramme de séquence Le responsable valide la formation des employés	17
2.6	Diagramme de séquence Le RH génère un rapport sur les écarts de compétences.....	17
2.7	Architecture globale de FormationPro sur AWS EKS	18
2.8	Architecture de déploiement AWS EKS (Terraform + GitHub Actions)	19
3.1	Interface de connexion FormationPro.....	22
3.2	Catalogue de formations.....	22
3.3	Vue des inscriptions et progression	23
3.4	Profil de compétences vue employé.....	23
3.5	Vue équipe suivi de progression (Manager).....	24
3.6	Formulaire de feedback formation	25
3.7	Tableau de bord des rapports (Manager/RH)	26
3.8	Pipeline CI/CD GitHub Actions (déploiement AWS EKS)	29
3.9	<u>Dashboard CloudWatch SkillTree-Dashboard-Avance</u>	31
3.10	État des alarmes CloudWatch.....	29

Liste des tableaux

1.1	Problèmes identifiés et impacts	2
1.2	Besoins non fonctionnels	4
1.3	Acteurs du système et droits associés	5
1.4	Technologies frontend	5
1.5	Technologies backend	6
1.6	Technologies de persistance	6
1.7	Technologies DevOps	7
1.8	Outils CI/CD et qualité	7
2.1	Rôles Scrum dans le projet	9
2.2	Matrice des risques projet	10
2.3	Backlog produit User Stories	11
2.4	Planification des sprints (Gantt simplifié)	12
2.5	Couches de l'architecture backend	19
3.1	Environnement de développement	21
3.2	Statuts des objectifs de formation	24
3.3	Rapports disponibles	25
3.4	Plan de tests fonctionnels	27
3.5	Plan de tests de sécurité (contrôle d'accès)	27
3.6	Ressources AWS provisionnées par Terraform	28
3.7	Jobs du pipeline GitHub Actions	29
3.8	Manifests Kubernetes et leur rôle	30

Introduction Générale

Dans un contexte économique en constante évolution, le développement des compétences est devenu un enjeu stratégique majeur pour les organisations. Cependant, la gestion des formations professionnelles reste souvent fragmentée et repose encore sur des outils hétérogènes tels que les fichiers Excel, les échanges par e-mail ou des applications non centralisées. Cette dispersion limite la visibilité globale sur les compétences disponibles, les parcours de formation suivis ainsi que l'évaluation de leur impact, ce qui complique le pilotage pour les différents acteurs de l'entreprise, notamment les employés, les managers et les ressources humaines.

Dans ce cadre, la problématique de ce projet consiste à concevoir et développer une plateforme numérique centralisée capable de gérer efficacement l'ensemble du cycle de vie de la formation professionnelle, depuis la consultation du catalogue et l'inscription aux formations jusqu'au suivi de la progression et à l'évaluation de leur impact. Cette solution doit également offrir une gestion adaptée aux différents profils utilisateurs à travers un système de rôles permettant un pilotage clair et sécurisé des activités de formation.

Le projet **FormationPro (Skill Tree)** a ainsi été conçu pour répondre à ces besoins en proposant une plateforme web complète et sécurisée. Celle-ci permet de centraliser le catalogue des formations et des compétences associées, de faciliter l'inscription et le suivi des employés, de formaliser les objectifs de formation définis par les managers avec des critères et des échéances, ainsi que de mesurer la satisfaction et l'impact des formations à travers des évaluations structurées. En complément, la solution fournit des rapports analytiques destinés aux managers et aux responsables des ressources humaines afin d'améliorer la prise de décision.

Le présent rapport est structuré en trois chapitres principaux :

- **Chapitre 1 Étude des besoins** : présentation du contexte, analyse des besoins fonctionnels et non fonctionnels, étude comparative des architectures logicielles et présentation des technologies retenues.
- **Chapitre 2 Conception** : description de la méthodologie Scrum, organisation des sprints, conception UML et architecture du système.
- **Chapitre 3 Réalisation et Tests** : environnement de développement, implémentation des fonctionnalités, stratégie de tests et déploiement sur infrastructure AWS EKS.

Chapitre 1

Étude des besoins

1.1 Présentation du contexte

1.1.1 Contexte du projet

Le projet **FormationPro**, également désigné sous le nom de code *Skill Tree*, naît du constat que la gestion des formations dans les organisations souffre d'une fragmentation structurelle. Les outils couramment utilisés (fichiers Excel, messagerie électronique, applications métier multiples et disparates) ne permettent pas d'assurer une visibilité cohérente sur les compétences, les parcours de formation et les performances individuelles.

Ce projet s'inscrit dans une démarche de **transformation numérique RH** : il vise à proposer une application web unifiée, multi-rôles, sécurisée et facilement déployable sur le cloud, capable de couvrir l'intégralité du cycle de gestion des formations, depuis le catalogue jusqu'à l'évaluation d'impact post-formation.

1.1.2 Problèmes identifiés

L'analyse du contexte fait ressortir les problèmes suivants :

Table 1.1. Problèmes identifiés et impacts

Problème	Impact
Faible visibilité sur les compétences	Impossibilité de détecter les écarts de compétences
Inscriptions non suivies	Progression inconnue, formations non finalisées
Pilotage managérial difficile	Objectifs d'équipe non formalisés ni mesurés
Mesure d'impact insuffisante	Retour sur investissement formation non évalué
Données dispersées	Aucune vue consolidée pour la décision RH

1.2 Analyse des besoins

1.2.1 Besoins fonctionnels

Les besoins fonctionnels sont organisés en neuf blocs thématiques, identifiés à partir de l'analyse des utilisateurs cibles.

1.2.1.1 BF1 Authentification et gestion des comptes

- Connexion par e-mail et mot de passe avec délivrance d'un jeton JWT.
- Réinitialisation du mot de passe : demande (*forgot password*) et réinitialisation via token.
- Inscription des comptes utilisateurs par le rôle RH.

1.2.1.2 BF2 Gestion des utilisateurs (RH)

- Création d'un utilisateur avec nom, prénom, e-mail, rôle et mot de passe.
- Consultation de la liste et du détail des utilisateurs.
- Mise à jour et suppression d'un utilisateur.

1.2.1.3 BF3 Catalogue de formations

- Consultation du catalogue par tout utilisateur authentifié.
- Création, édition et suppression d'une formation par le rôle RH.
- Association de compétences à une formation (*Training Skills*).

1.2.1.4 BF4 Inscriptions et progression

- Inscription d'un employé à une formation.
- Consultation des inscriptions personnelles.
- Mise à jour du statut et du pourcentage de progression.

1.2.1.5 BF5 Compétences (Skill Tree)

- Consultation du référentiel de compétences.
- Visualisation du profil de compétences d'un employé.
- Mise à jour du niveau de compétence par manager ou RH.

1.2.1.6 BF6 Objectifs de formation

- Assignation d'un objectif de formation à un employé avec deadline et critères.
- Suivi du statut : PENDING, IN_PROGRESS, ACHIEVED, OVERDUE.
- Vue *team progress* pour manager/RH.

1.2.1.7 BF7 Feedback et évaluation d'impact

- Feedback employé : note de 1 à 5 et commentaire (un seul feedback par formation/employé).
- Évaluation d'impact manager/RH : score d'impact de 1 à 5 et commentaire.

1.2.1.8 BF8 Notifications

- Persistance des notifications en base de données.
- Récupération des notifications non lues et du compteur associé.

1.2.1.9 BF9 Rapports

- Rapport des formations (statistiques générales).
- Rapport de *skills gap* (écarts de compétences).
- Rapport des formations les plus suivies (*top trainings*).

1.2.2 Besoins non fonctionnels

Table 1.2. Besoins non fonctionnels

Catégorie	Description
Sécurité	Authentification stateless (JWT), contrôle des rôles (RBAC), mots de passe hachés (BCrypt)
Performance	API REST paginable et filtrable, temps de réponse acceptable sous charge
Fiabilité	Contraintes d'unicité (1 inscription/formation/utilisateur, 1 feedback, 1 évaluation)
Traçabilité	Horodatage des créations et mises à jour, source de modification des compétences
Portabilité	Images Docker hébergées sur Amazon ECR, déployées sur AWS EKS via Terraform
Maintenabilité	Architecture en couches (Controller / Service / Repository / DTO)
Qualité code	Analyse statique SonarCloud intégrée au pipeline CI/CD
Disponibilité	Exposition via AWS Application Load Balancer (internet-facing) avec healthchecks

1.2.3 Acteurs du système

Le système distingue quatre acteurs principaux :

Table 1.3. Acteurs du système et droits associés

Acteur	Rôle technique	Périmètre d'accès
Visiteur		Pages publiques (login, reset password)
Employé	EMPLOYEE	Catalogue, inscriptions, progression, feedback, compétences
Manager	MANAGER	Employé + équipe, objectifs, évaluations, rapports
RH	HR	Manager + administration utilisateurs, gestion formations, reporting global

1.3 Technologies et outils

1.3.1 Frontend

Table 1.4. Technologies frontend

Technologie	Version	Rôle et justification
React	18+	Bibliothèque de construction d'interfaces réactives basée sur les composants. Large écosystème et forte adoption industrielle.
TypeScript	5+	Typage statique pour JavaScript. Améliore la robustesse, la maintenabilité et l'expérience de développement.
Vite	5+	Outil de build ultra-rapide (ESM natif). Remplace Create React App pour sa rapidité de démarrage et de HMR.
Tailwind CSS	3+	Framework CSS utilitaire. Permet un design cohérent et rapide sans couche CSS personnalisée complexe.
shadcn/ui		Composants UI accessibles et personnalisables basés sur Radix UI et Tailwind CSS.
React Router	6+	Routage côté client (SPA). Gestion des routes publiques et protégées.

1.3.2 Backend

Table 1.5. Technologies backend

Technologie	Version	Rôle et justification
Java	17 LTS	Langage principal. Version LTS offrant un support long terme et les dernières fonctionnalités du langage.
Spring Boot	3.x	Framework applicatif Java. Auto-configuration, IoC, gestion des dépendances. Standard de l'industrie pour les API REST.
Spring Security	6.x	Sécurisation de l'application : filtres d'authentification, autorisations par rôle, CORS.
Spring Data JPA		Couche d'abstraction de l'accès aux données via Hibernate/JPA.
JWT (JJWT)	0.12+	Gestion des jetons d'authentification stateless (HS256).
BCrypt		Hachage des mots de passe (coût adaptatif).
Spring Actuator		Exposition des endpoints de santé (/health, /info) pour les healthchecks Kubernetes.
Maven	3.9+	Outil de build et gestion des dépendances.

1.3.3 Base de données

Table 1.6. Technologies de persistance

Technologie	Version	Rôle et justification
MariaDB	10.11	SGBD relationnel déployé en StatefulSet Kubernetes sur EKS. Robustesse, contraintes d'intégrité (unicité, clés étrangères), compatibilité MySQL.
Hibernate		ORM (Object-Relational Mapping). Génération et migration du schéma, requêtes JPQL.

1.3.4 DevOps et déploiement

Table 1.7. Technologies DevOps

Technologie	Rôle
Docker	Conteneurisation des services (backend, frontend). Images construites dans le pipeline CI/CD.
Amazon ECR	Registre d'images Docker managé AWS. Stockage des images taguées par SHA de commit (GITHUB_SHA::8) et latest.
Amazon EKS	Cluster Kubernetes managé sur AWS. Orchestration des workloads backend, frontend et base de données dans le namespace skilltree.
AWS ALB	Application Load Balancer internet-facing. Routage HTTP/HTTPS vers les services frontend et backend via l'Ingress Controller AWS.
AWS Load Balancer Controller	Déployé via Helm par Terraform (alb-controller.tf). Gère le cycle de vie des ALB depuis les ressources Ingress Kubernetes.
Terraform	Infrastructure as Code. Provisionne le VPC, les subnets, le cluster EKS, le NodeGroup, les repos ECR et l'add-on aws-ebs-csi-driver.
Amazon S3	Backend de stockage de l'état Terraform (bucket créé au <i>bootstrap</i> du pipeline).
Helm	Déploiement du AWS Load Balancer Controller via chart Helm dans Terraform.

1.3.5 CI/CD et qualité

Table 1.8. Outils CI/CD et qualité

Outil	Rôle
GitHub Actions	Pipeline CI/CD multi-jobs : bootstrap, détection de changements, analyse SonarCloud, Terraform plan/apply, build et push images ECR, déploiement EKS.
SonarCloud	Analyse statique de la qualité du code backend (bugs, vulnérabilités, dette technique) via Maven dans le job sonar.

Conclusion du chapitre

L'analyse des besoins a permis de définir un périmètre fonctionnel complet, structuré autour de neuf blocs de fonctionnalités couvrant l'ensemble du cycle de vie de la formation professionnelle. L'étude comparative des architectures a conduit au choix d'une architecture client–serveur en couches, combinant la maîtrisabilité d'un monolithe structuré et la séparation claire des responsabilités entre frontend et backend. La stack technologique retenue Spring Boot, React/TypeScript, MariaDB, AWS EKS, Terraform, GitHub Actions allie maturité industrielle, productivité et capacités cloud avancées. Ces choix constituent le socle sur lequel repose la conception présentée au chapitre suivant.

Chapitre 2

Conception

Ce chapitre décrit la démarche de conception du projet FormationPro. Il présente la méthodologie Agile Scrum adoptée, l'organisation des sprints, les diagrammes UML structurant le système et l'architecture globale retenue.

2.1 Méthodologie Agile Scrum

2.1.1 Présentation de Scrum

La méthodologie **Scrum** est un cadre de gestion de projet agile fondé sur des itérations courtes et régulières appelées **sprints**. Elle favorise la livraison incrémentale de valeur, la collaboration étroite entre les parties prenantes et l'adaptation continue aux changements.

2.1.2 Cérémonies Scrum

- **Sprint Planning** : sélection des *user stories* du sprint, estimation (points de story), définition du *Sprint Goal*.
- **Daily Scrum** : réunion quotidienne de 10 à 15 minutes (avancement, obstacles).
- **Sprint Review** : démonstration des incréments livrés aux parties prenantes.
- **Rétrospective** : identification des axes d'amélioration du processus.

2.1.3 Definition of Done (DoD)

Une *user story* est considérée **Done** si :

- Les fonctionnalités sont livrées et démontrables.
- Les contrôles d'accès par rôle sont respectés.
- Les validations métier sont implémentées.
- Les tests fonctionnels clés sont passés avec succès.
- Aucune régression n'est visible sur les parcours principaux.

2.1.4 Gestion des risques

Table 2.2. Matrice des risques projet

Risque	Prob.	Impact	Mitigation
Incohérences des règles d'accès par rôle	Moyenne	Élevé	Matrice RBAC + tests de sécurité systématiques
Complexité du modèle de données	Moyenne	Moyen	Itération par sprint, migrations simples
Régressions UI lors des ajouts de features	Moyenne	Moyen	Scénarios de non-régression documentés
Données de démo insuffisantes	Faible	Moyen	Seeder applicatif + jeux d'essai définis
Déploiement instable (EKS)	Moyenne	Élevé	Détection et import automatique des ressources AWS existantes, healthchecks et rollout status dans le pipeline
Coûts AWS non maîtrisés	Moyenne	Moyen	NodeGroup dimensionné au minimum, destroy job manuel dans le pipeline

2.2 Backlog produit et user stories

Le backlog produit recense l'ensemble des *user stories* priorisées pour le projet.

Table 2.3. Backlog produit User Stories

ID	Acteur	User Story
US-A1	Visiteur	En tant que Visiteur, je veux me connecter afin d'accéder au tableau de bord.
US-A2	Visiteur	En tant que Visiteur, je veux réinitialiser mon mot de passe afin de récupérer l'accès.
US-E1	Employé	En tant qu'Employé, je veux consulter le catalogue afin de choisir une formation.
US-E2	Employé	En tant qu'Employé, je veux m'inscrire à une formation afin de la suivre.
US-E3	Employé	En tant qu'Employé, je veux voir mes inscriptions et ma progression afin de suivre mon parcours.
US-E4	Employé	En tant qu'Employé, je veux laisser un feedback après une formation afin d'évaluer la qualité.
US-E5	Employé	En tant qu'Employé, je veux consulter mon profil de compétences afin d'identifier mes axes d'amélioration.
US-M1	Manager	En tant que Manager, je veux consulter l'équipe afin de suivre la progression de mes collaborateurs.
US-M2	Manager	En tant que Manager, je veux assigner des objectifs de formation afin d'aligner l'équipe sur les besoins.
US-M3	Manager	En tant que Manager, je veux évaluer l'impact d'une formation afin de mesurer le retour.
US-M4	Manager	En tant que Manager, je veux consulter des rapports afin de prendre des décisions éclairées.
US-RH1	RH	En tant que RH, je veux gérer les utilisateurs afin d'administrer les accès et les rôles.
US-RH2	RH	En tant que RH, je veux gérer le catalogue de formations afin d'aligner l'offre sur la stratégie.
US-RH3	RH	En tant que RH, je veux consulter des rapports globaux afin de piloter la montée en compétences.

2.3 Organisation des sprints

Le projet est découpé en **quatre sprints de deux semaines** chacun, soit un total de huit semaines de développement.

Table 2.4. Planification des sprints (Gantt simplifié)

Sprint	Durée	Focus	Livrables clés
Sprint 1	Semaines 1–2	Auth + Sécurité + Utilisateurs	JWT, RBAC, CRUD users, écrans auth
Sprint 2	Semaines 3–4	Catalogue + Compétences + Inscriptions	CRUD formations/skills, inscriptions, écrans catalogue
Sprint 3	Semaines 5–6	Objectifs + Équipe + Feedback + Notifications	Goals, team progress, feedback, notifications
Sprint 4	Semaines 7–8	Évaluations + Rapports + Déploiement AWS	Évaluations impact, rapports, EKS + Terraform, CI/CD

2.3.1 Sprint 1 Socle : Authentification, Utilisateurs et Sécurité

Objectif

Sécuriser l'accès à l'application et mettre en place l'administration de base des utilisateurs.

User Stories

US-A1, US-A2, US-RH1.

Tâches réalisées

- Configuration de Spring Security en mode stateless.
- Implémentation du filtre JWT (JwtAuthenticationFilter).
- Développement des endpoints d'authentification : /api/auth/login, /api/auth/forgot-password, /api/auth/reset-password.
- CRUD complet des utilisateurs avec validation et contrôle de rôle (HR).
- Configuration CORS pour autoriser le frontend.
- Développement du frontend : pages Login, Signup, Forgot Password, Reset Password, stockage du token JWT.

Livrables

JWT opérationnel, RBAC configuré, CRUD utilisateurs, interfaces d'authentification.

Critères d'acceptation

- Un utilisateur authentifié accède au dashboard.
- Le rôle HR peut créer et supprimer un compte.
- La réinitialisation de mot de passe fonctionne via token.

2.3.2 Sprint 2 Catalogue, Inscriptions et Compétences

Objectif

Offrir le parcours employé complet : choisir une formation, s'inscrire et suivre sa progression.

User Stories

US-E1, US-E2, US-E3, US-E5, US-RH2.

Tâches réalisées

- CRUD formations (RH) avec association de compétences.
- CRUD compétences (Skill referential).
- Endpoint d'inscription : POST /api/trainings/{id}/enroll.
- Contrainte d'unicité (1 inscription/formation/utilisateur).
- Consultation des inscriptions et mise à jour de la progression.
- Écrans frontend : Catalogue, Détail formation, Mes inscriptions, Profil compétences.

Livrables

CRUD formations et compétences, système d'inscription, affichage de la progression.

Critères d'acceptation

- Une seule inscription par utilisateur/formation.
- Progression affichée et mise à jour.
- Compétences associées aux formations visibles.

2.3.3 Sprint 3 Pilotage Manager : Équipe, Objectifs, Feedback et Notifications

Objectif

Doter les managers et RH d'outils de pilotage et favoriser l'engagement des employés.

User Stories

US-M1, US-M2, US-E4 et notifications.

Tâches réalisées

- Gestion des objectifs de formation (TrainingGoal) avec deadline et statut.
- Endpoint GET /api/manager/team-progress pour la vue d'équipe.
- Feedback employé : note + commentaire, unicité garantie.
- Persistance des notifications + badge compteur de non-lues.
- Écrans frontend : Équipe, Objectifs, Feedback, badge notifications.

Livrables

Module objectifs, vue équipe, feedback, système de notifications.

Critères d'acceptation

- Le manager voit l'avancement de son équipe.
- Un objectif possède une deadline et un statut évolutif.
- Un seul feedback par formation/utilisateur.

2.3.4 Sprint 4 Mesure d'impact, Rapports et Déploiement AWS

Objectif

Mesurer l'efficacité des formations, fournir la couche décisionnelle et déployer l'application sur AWS EKS.

User Stories

US-M3, US-M4, US-RH3, stabilisation et déploiement cloud.

Tâches réalisées

- Évaluations d'impact (PerformanceEvaluation) : score + commentaire manager.
- Rapports : formations (/api/reports/trainings), skills gap (/api/reports/skills-gap), top formations (/api/reports/top-trainings).
- Écriture des manifests Kubernetes (k8s-manifests/) : namespace/configmap, MariaDB StatefulSet, Deployment backend, Deployment frontend, Ingress ALB.
- Écriture des fichiers Terraform : VPC, EKS, NodeGroup, ECR, aws-ebs-csi-driver, AWS Load Balancer Controller.
- Pipeline CI/CD GitHub Actions multi-jobs (.github/workflows/deploy.yml).
- Données de démonstration (seeder).

Livrables

Évaluations impact, rapports RH/manager, infrastructure AWS EKS opérationnelle.

Critères d'acceptation

- Une seule évaluation par employé/formation.
- Rapports accessibles selon le rôle.
- Application accessible via l'URL de l'ALB après un *push* sur main.

2.4 Conception UML

2.4.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation illustre les interactions entre les acteurs du système et les fonctionnalités offertes par FormationPro.

de cas d'utilisation.png de cas d'utilisation.bb

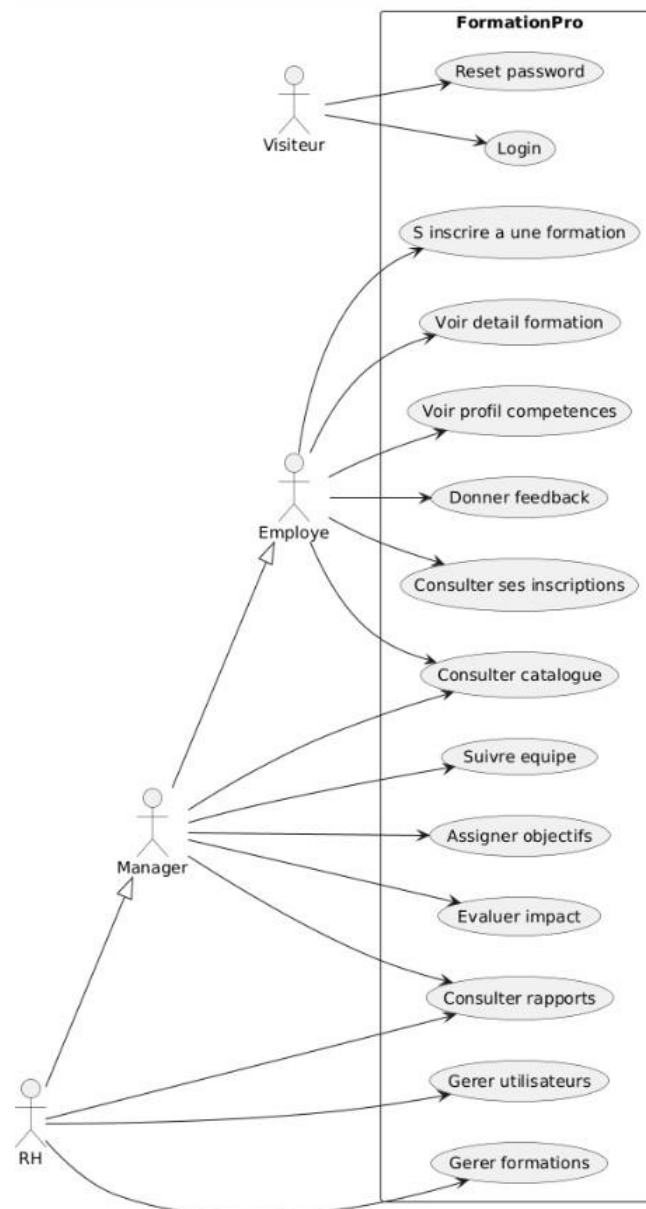


Figure 2.1. Diagramme de cas d'utilisation FormationPro

2.4.2 Diagramme de classes

Le diagramme de classes représente les entités du domaine métier et leurs relations.

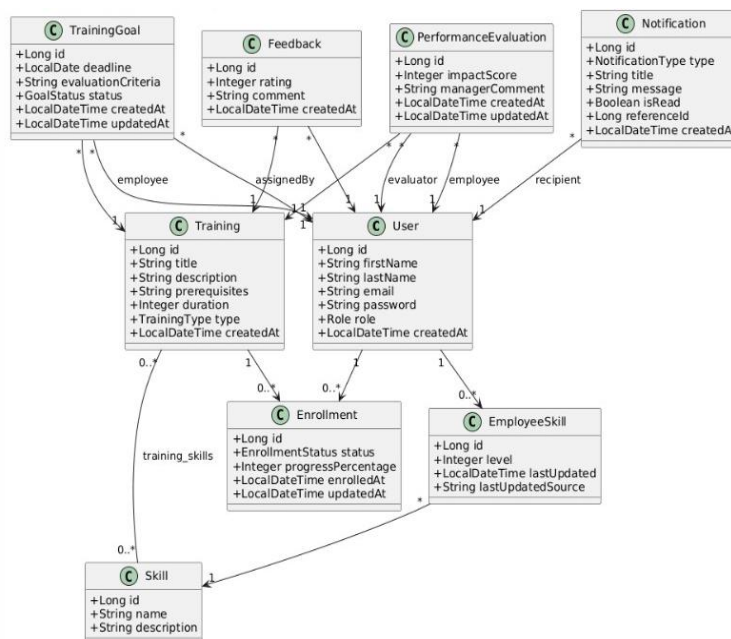


Figure 2.2. Diagramme de classes Domaine métier FormationPro

2.4.3 Diagramme de séquence Authentification (Login)

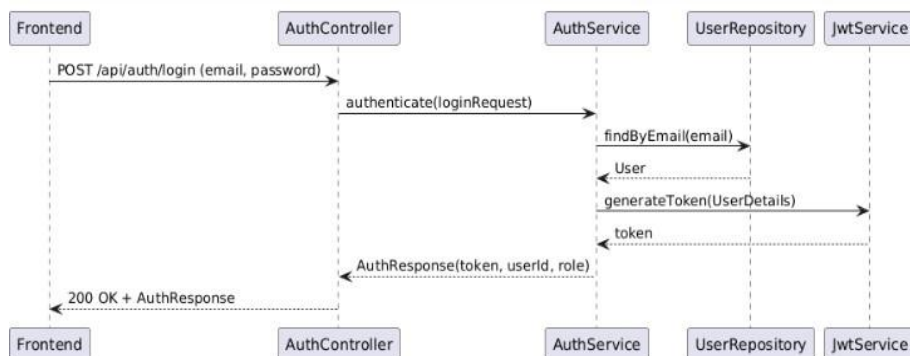


Figure 2.3. Diagramme de séquence Flux de connexion JWT

2.4.4 Diagramme de séquence Le responsable valide la formation des employés

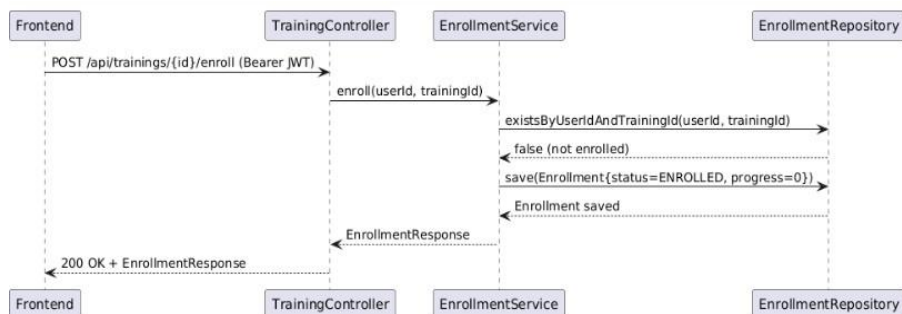


Figure 2.4. Diagramme de séquence Inscription à une formation

2.4.5 Diagramme de séquence Le responsable valide la formation des employés

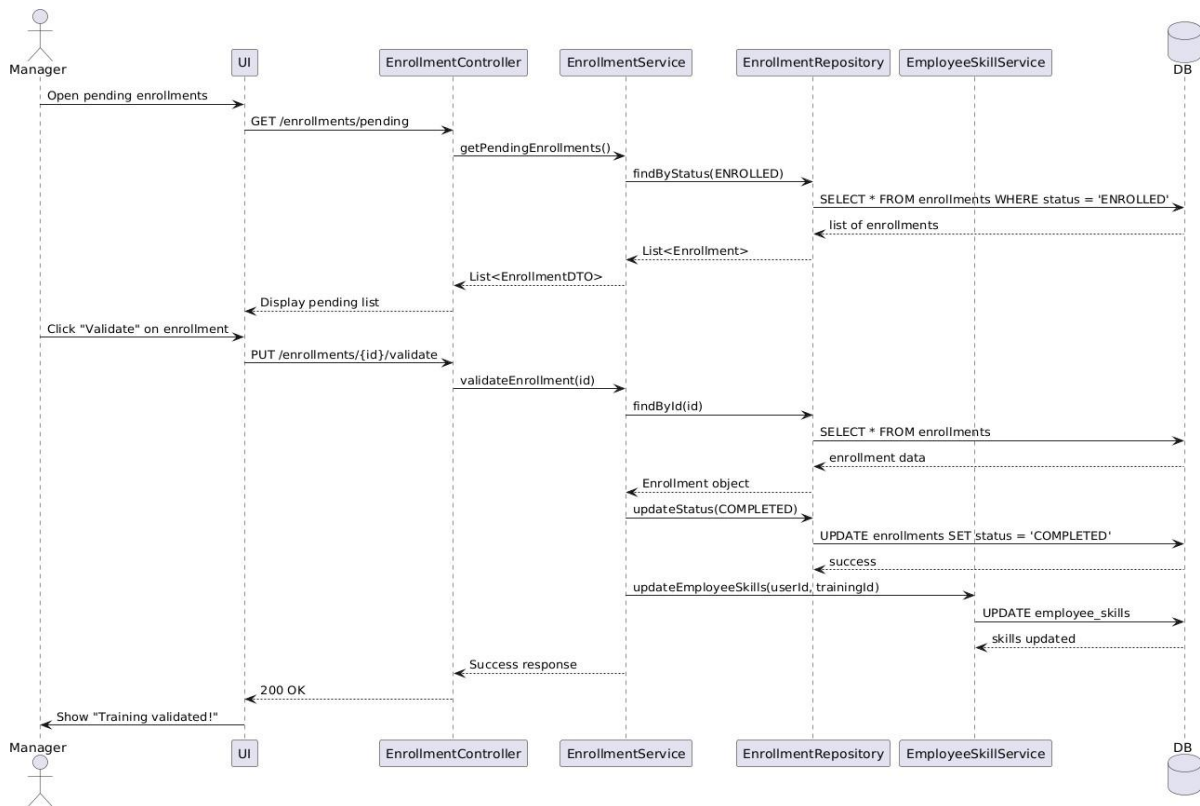


Figure 2.5. Diagramme de séquence Le responsable valide la formation des employés

2.4.6 Diagramme de séquence le RH génère un rapport sur les écarts de compétences

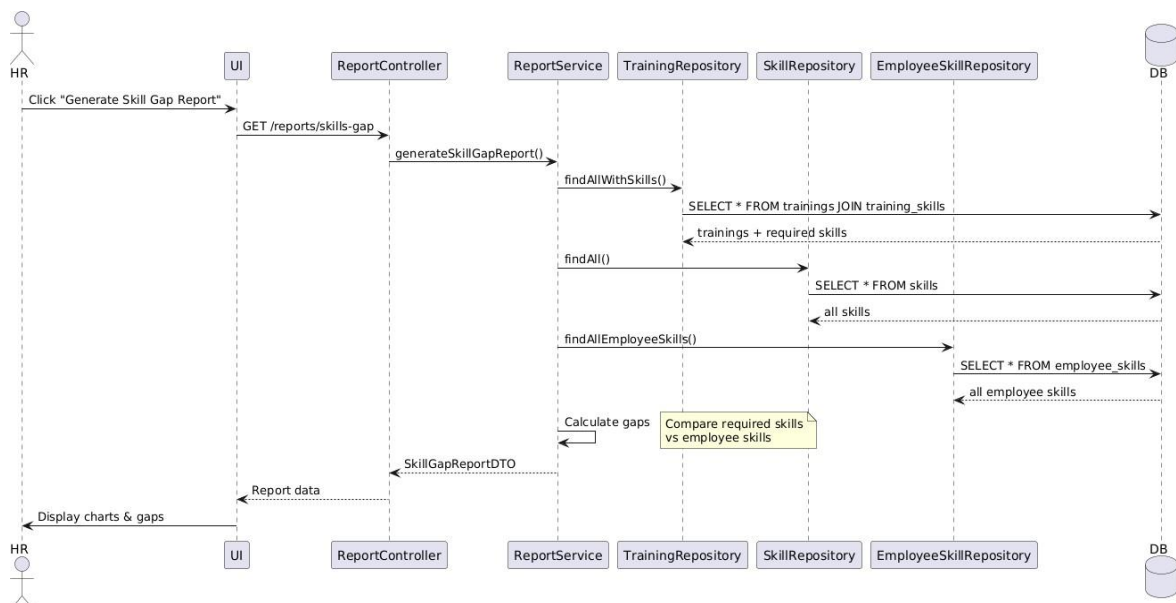


Figure 2.6. Diagramme de séquence Le RH génère un rapport sur les écarts de compétences

2.5 Architecture du système

2.5.1 Architecture globale

L'application FormationPro adopte une architecture **client-serveur à trois niveaux** déployée sur AWS :

1. **Niveau présentation** : application React/TypeScript (SPA) exécutée dans le navigateur, servie par des pods Nginx sur EKS, exposée via l'AWS ALB.
2. **Niveau applicatif** : API REST Spring Boot (Java 17) exécutée dans des pods sur EKS, exposant les endpoints métier sécurisés par JWT.
3. **Niveau données** : base de données MariaDB déployée en StatefulSet dans le cluster EKS.

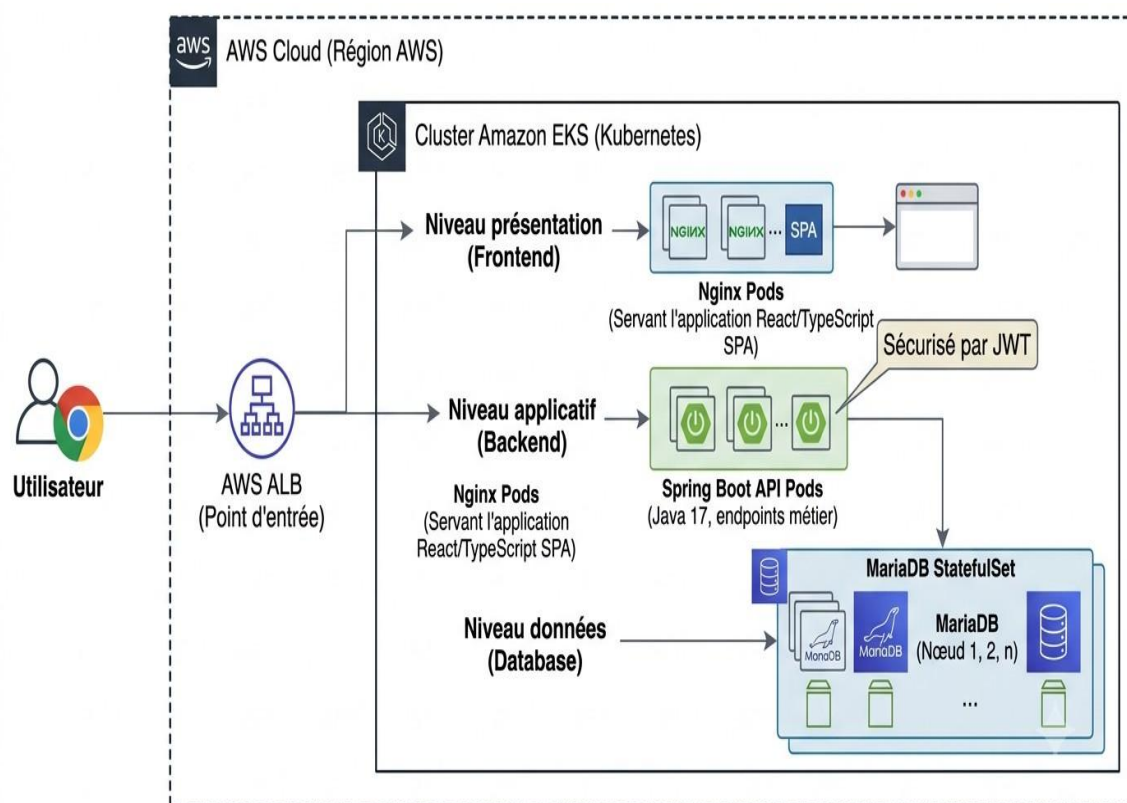


Figure 2.7. Architecture globale de FormationPro sur AWS EKS

2.5.2 Architecture backend en couches

Le backend Spring Boot est organisé en couches strictement séparées :

Table 2.5. Couches de l'architecture backend

Couche	Responsabilité
Controller	Exposition des endpoints REST, validation de la requête HTTP, délégation au service.
Service	Implémentation des règles métier, orchestration des repositories, calculs.
Repository	Accès à la base de données via Spring Data JPA (interfaces JpaRepository).
DTO	Objets de transfert découplant le modèle REST du modèle d'entité (sécurité + stabilité).
Entity	Mapping objet-relationnel (Hibernate), contraintes JPA.
Security	Filtres Spring Security, JwtAuthenticationFilter, règles d'autorisation.

2.5.3 Architecture cloud AWS EKS

En environnement de production, l'application est déployée sur un cluster **Amazon EKS** dont l'infrastructure est entièrement provisionnée par **Terraform**.

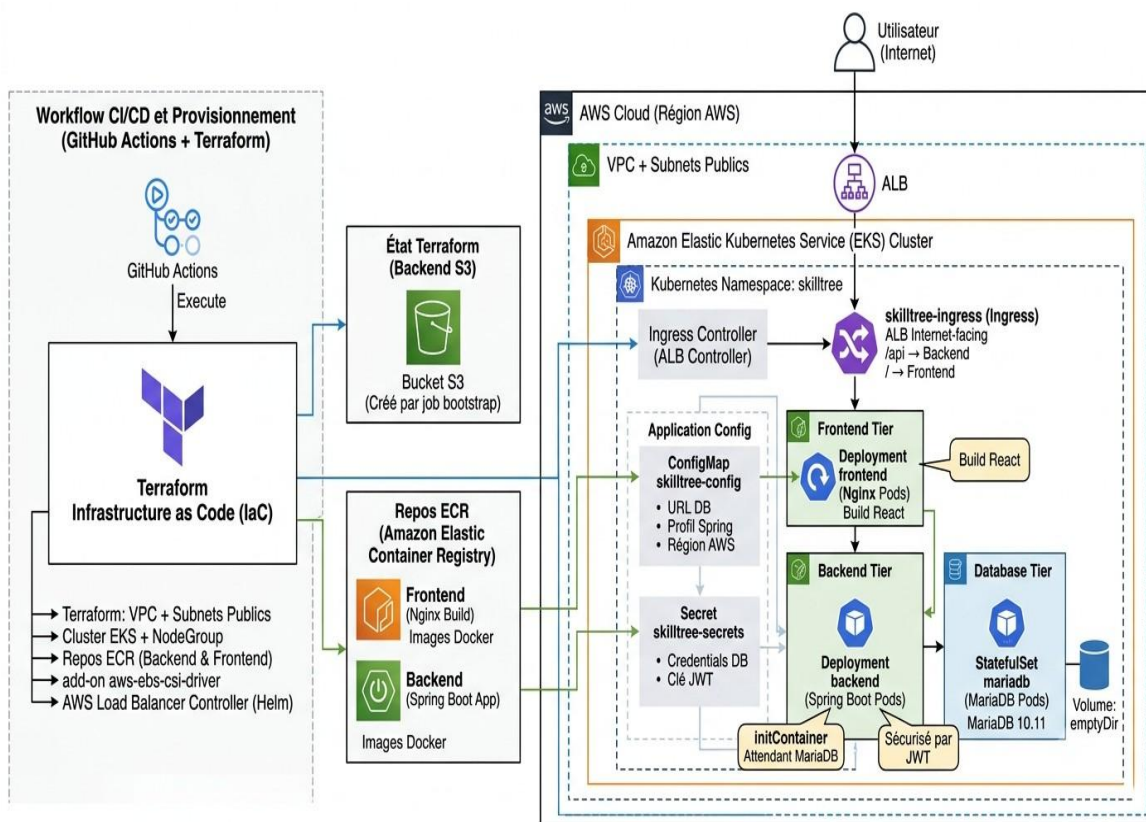


Figure 2.8. Architecture de déploiement AWS EKS (Terraform + GitHub Actions)

Les composants provisionnés et déployés sont :

- **Terraform** : VPC + subnets publics, cluster EKS + NodeGroup, repos ECR (backend & frontend), add-on aws-ebs-csi-driver, AWS Load Balancer Controller (Helm).
- **Namespace** skilltree : isolation des ressources applicatives.
- **ConfigMap** skilltree-config : variables d'environnement (URL DB, profil Spring, région AWS...).
- **Secret** skilltree-secrets : credentials DB et clé JWT (créés/mis à jour par le pipeline).
- **Deployment frontend** : pods Nginx servant le build React (image ECR).
- **Deployment backend** : pods Spring Boot avec initContainer attendant MariaDB (image ECR).
- **StatefulSet mariadb** : MariaDB 10.11 (volume emptyDir).
- **Ingress** skilltree-ingress : ALB internet-facing, routage /api backend, / frontend.
- **État Terraform** : persisté dans un bucket **Amazon S3** créé lors du job *bootstrap*.

2.5.4 Flux de données et sécurité

Flux d'authentification

1. Le frontend envoie POST /api/auth/login avec l'e-mail et le mot de passe.
2. Spring Boot valide les identifiants (BCrypt) et génère un JWT signé (HS256).
3. Le frontend stocke le token et l'inclut dans l'en-tête Authorization: Bearer <token> de chaque requête.
4. Le filtre JwtAuthenticationFilter intercepte chaque requête, valide le token et charge le contexte de sécurité.

Conclusion du chapitre

La conception de FormationPro s'appuie sur une méthodologie Scrum rigoureuse structurée en quatre sprints, garantissant une livraison incrémentale et validée de l'application. Les diagrammes UML (cas d'utilisation, classes, séquences) fournissent un référentiel de conception clair et cohérent. L'architecture client-serveur en couches, couplée à un déploiement cloud sur Amazon EKS entièrement automatisé par Terraform et GitHub Actions, constitue un socle technique robuste, scalable et reproductible. Le chapitre suivant décrit la réalisation concrète de ces choix de conception.

Chapitre 3

Réalisation et Tests

3.1 Environnement de développement

3.1.1 Outils et frameworks

Table 3.1. Environnement de développement

Outil	Version	Usage
IntelliJ IDEA / VS Code		IDE de développement backend (Java) et frontend (TypeScript).
Node.js + npm	20+ LTS	Exécution de l'environnement frontend Vite.
Java JDK	17 LTS	Compilation et exécution du backend Spring Boot.
Maven	3.9+	Build et gestion des dépendances backend.
Docker Desktop		Conteneurisation locale, build des images.
Terraform	1.x	Provisionnement de l'infrastructure AWS (EKS, VPC, ECR).
AWS CLI	2+	Interaction avec les services AWS, configuration de kubeconfig.
kubectl		Interaction avec le cluster Amazon EKS.
Helm	3+	Déploiement du AWS Load Balancer Controller.
Postman / Thunder Client		Test des endpoints REST de l'API.
Git + GitHub		Gestion de versions et hébergement du code source.

3.2 Implémentation

3.2.1 Module Authentification et Sécurité JWT

Le module d'authentification est au cur de la sécurité de l'application. Il repose sur les principes suivants :

- **Stateless** : aucune session côté serveur n'est maintenue.

- **JWT Bearer** : le token est transmis dans l'en-tête HTTP Authorization.
- **BCrypt** : les mots de passe sont stockés sous forme de hash adaptatif.
- **Secret injecté** : la clé JWT est stockée dans un **Secret Kubernetes** (skilltree-secrets) et injectée comme variable d'environnement, jamais en dur dans le code.

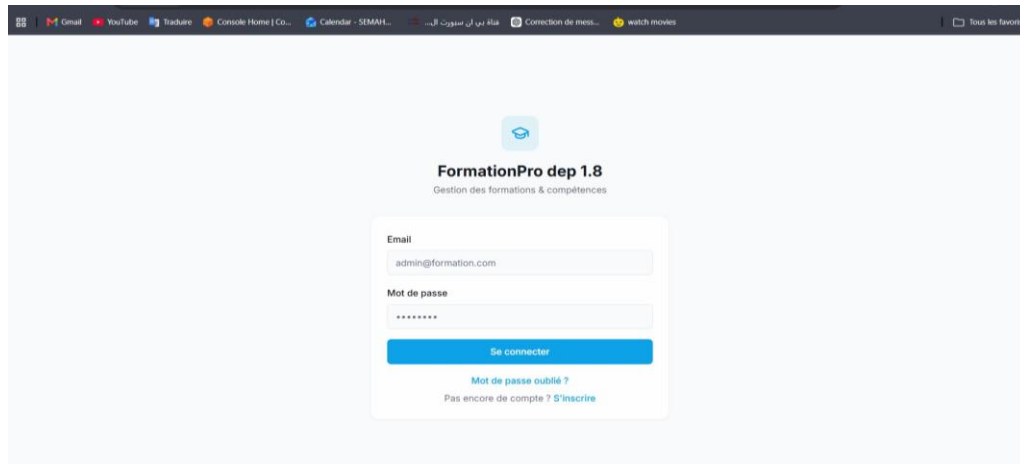
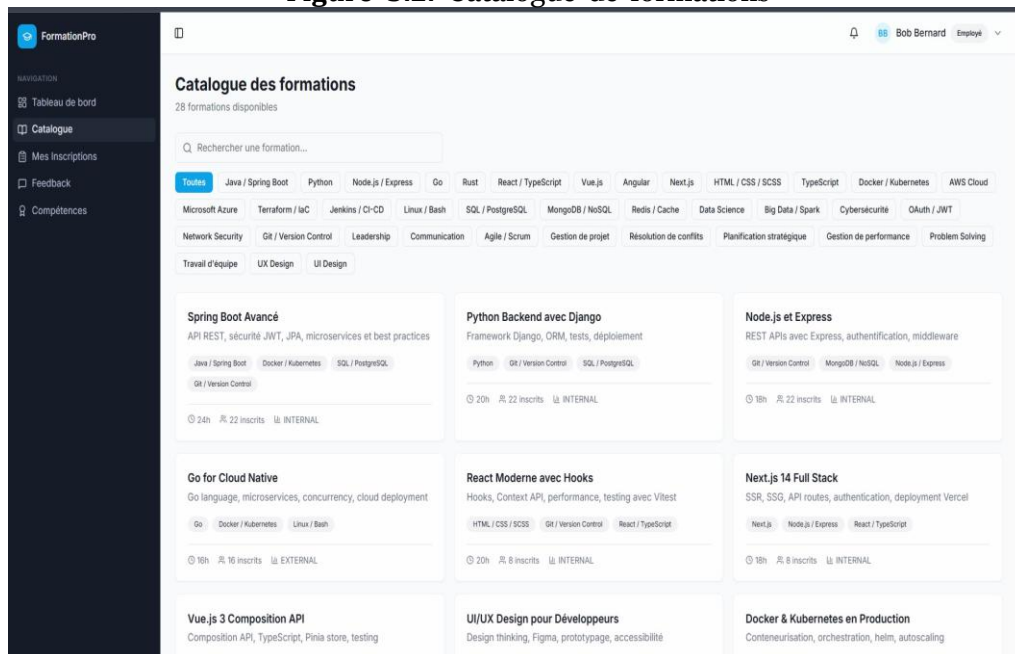


Figure 3.1. Interface de connexion FormationPro

3.2.2 Module Catalogue de Formations

Le catalogue permet à tous les utilisateurs authentifiés de consulter les formations disponibles. Les RH peuvent créer, modifier et supprimer des formations.

Figure 3.2. Catalogue de formations



3.2.3 Module Inscriptions et Progression

L'inscription garantit l'unicité via une contrainte composite sur (user_id, training_id).

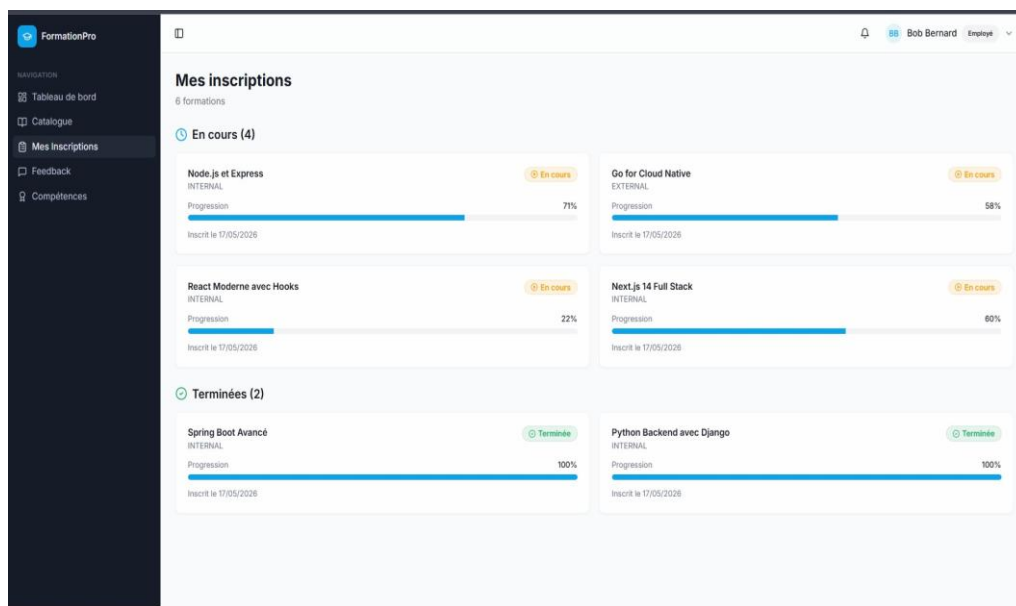


Figure 3.3. Vue des inscriptions et progression

3.2.4 Module Compétences (Skill Tree)

Le référentiel de compétences permet de cartographier les aptitudes de chaque employé et d'identifier les écarts avec les compétences cibles.

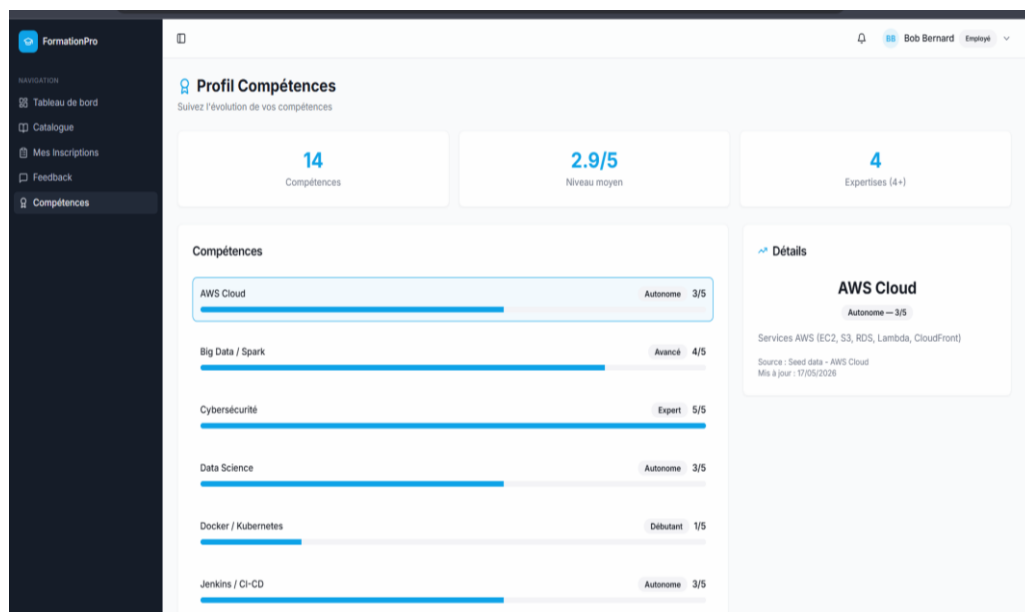


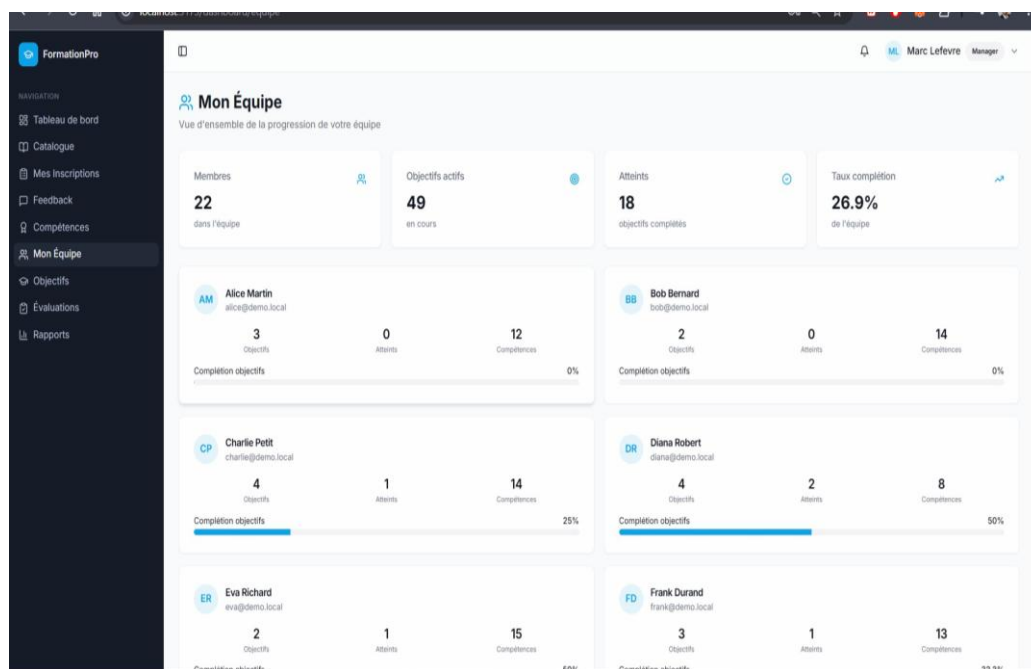
Figure 3.4. Profil de compétences vue employé

3.2.5 Module Objectifs et Pilotage Manager

Les objectifs de formation permettent aux managers d'assigner formellement à un employé une formation à compléter avant une deadline, avec des critères d'évaluation documentés.

Table 3.2. Statuts des objectifs de formation

Statut	Signification
PENDING	Objectif assigné, formation non commencée.
IN_PROGRESS	Formation en cours de suivi.
ACHIEVED	Formation complétée, objectif atteint.
OVERDUE	Deadline dépassée, objectif non atteint.

**Figure 3.5.** Vue équipe suivi de progression (Manager)

3.2.6 Module Feedback et Évaluations d'impact

- **Feedback employé** : note de 1 à 5 étoiles et commentaire. Contrainte d'unicité : un seul feedback par formation/employé.
- **Évaluation manager** : score d'impact de 1 à 5 et commentaire. Contrainte d'unicité : une seule évaluation par formation/employé.

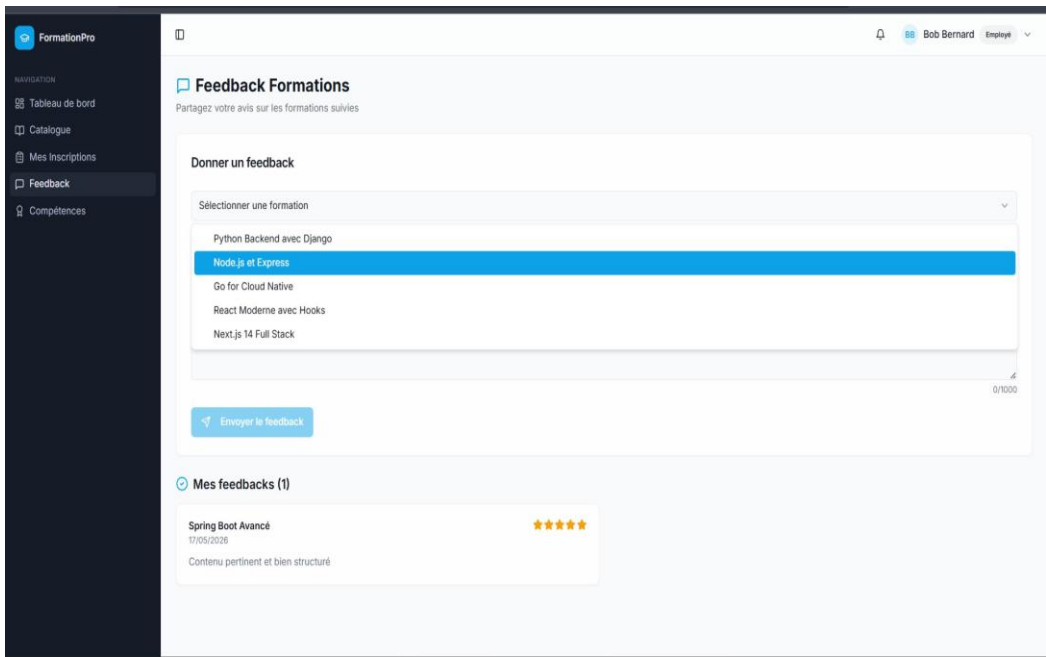


Figure 3.6. Formulaire de feedback formation

3.2.7 Module Rapports

Trois rapports sont disponibles pour les rôles Manager et HR :

Table 3.3. Rapports disponibles

Rapport	Endpoint	Contenu
Formations	GET /api/reports/trainings	Statistiques globales des formations (nombre d'inscrits, progression moyenne, feedbacks).
Skills Gap	GET /api/reports/skills-gap	Écarts entre compétences détenues et compétences cibles par employé.
Top Formations	GET /api/reports/top-trainings	Classement des formations les plus suivies et les mieux notées.

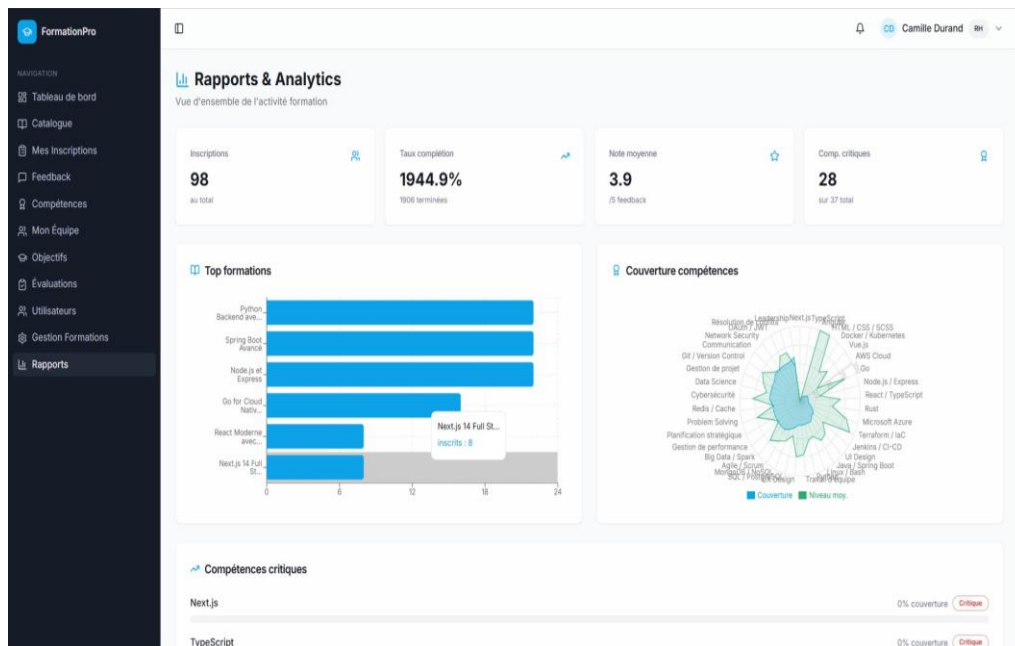


Figure 3.7. Tableau de bord des rapports (Manager/RH)

3.3 Tests

3.3.1 Stratégie de tests

La stratégie de tests du projet FormationPro couvre quatre niveaux complémentaires :

- **Tests fonctionnels** : validation des fonctionnalités via l'IHM et/ou Postman.
- **Tests de sécurité** : vérification du contrôle d'accès par rôle.
- **Tests d'intégration** : vérification des contraintes base de données et des relations JPA.
- **Tests de non-régression** : scénarios couvrant les parcours principaux.

3.3.2 Tests fonctionnels

Table 3.4. Plan de tests fonctionnels

ID	Scénario	Résultat attendu	Statut
TF-AUTH-01	Connexion avec identifiants valides	HTTP 200, token JWT non vide	✓
TF-AUTH-02	Connexion avec mot de passe incorrect	HTTP 401, message d'erreur	✓
TF-USERS-01	Création utilisateur (rôle HR)	Utilisateur créé, email unique	✓
TF-TRAIN-01	Consultation catalogue (auth)	Liste formations affichée, pas de 401	✓
TF-ENR-01	Inscription à une formation	Inscription créée (statut ENROLLED, progression 0)	✓
TF-ENR-02	Double inscription à la même formation	Rejet 409, aucune duplication	✓
TF-FDB-01	Double feedback sur même formation	Rejet sur le 2 ^e envoi	✓
TF-GOAL-01	Assigner un objectif (Manager/RH)	Objectif créé statut PENDING	✓
TF-REP-01	Accès aux rapports (Manager/RH)	HTTP 200, données cohérentes	✓

3.3.3 Tests de sécurité

Table 3.5. Plan de tests de sécurité (contrôle d'accès)

ID	Scénario	Résultat attendu	Statut
TS-SEC-01	Appel endpoint protégé sans token	HTTP 401	✓
TS-SEC-02	Création formation avec rôle EMPLOYEE	HTTP 403	✓
TS-SEC-03	Token JWT altéré/expiré	HTTP 401	✓
TS-SEC-04	Accès rapport avec rôle EMPLOYEE	HTTP 403	✓

3.3.4 Tests d'intégration

Les tests d'intégration vérifient la cohérence entre les couches applicatives et la base de données :

- **Unicité des inscriptions** : contrainte (user_id, training_id) UNIQUE sur

Enrollment.

- **Unicité des feedbacks** : contrainte (user_id, training_id) UNIQUE sur Feedback.
- **Unicité des évaluations** : contrainte (employee_id, training_id) UNIQUE sur PerformanceEvaluation.
- **Relation Training–Skill** : association ManyToMany via table de jointure training_skills.
- **Notifications** : persistance correcte, requêtes *unread* et compteur.

3.4 Déploiement

3.4.1 Infrastructure AWS (Terraform)

L’infrastructure de production est entièrement définie et provisionnée en tant que code (**Infrastructure as Code**) via **Terraform**, dont l’état est persisté dans un bucket **Amazon S3** créé automatiquement lors du job *bootstrap* du pipeline.

Table 3.6. Ressources AWS provisionnées par Terraform

Ressource	Description
VPC + subnets publics	Réseau isolé avec subnets publics pour les nuds EKS.
Cluster EKS	Cluster Kubernetes managé AWS.
NodeGroup EKS	Groupe de nuds EC2 exécutant les pods applicatifs.
ECR backend	Registre d’images Docker pour le backend Spring Boot.
ECR frontend	Registre d’images Docker pour le frontend Nginx/-React.
Add-on aws-ebs-csi-driver	Driver CSI EBS pour les volumes persistants Kubernetes.
AWS Load Balancer Controller	Contrôleur Kubernetes gérant le cycle de vie des ALB (déployé via Helm dans alb-controller.tf).
Amazon S3 (état Terraform)	Bucket de stockage de l’état Terraform (créé au bootstrap).

Gestion des ressources AWS existantes

Le job *bootstrap* du pipeline détecte les ressources AWS déjà existantes (cluster EKS, repos ECR, VPC/subnets. . .) et les **importe** dans l’état Terraform avant le plan/apply, évitant ainsi les conflits lors de réexecutions successives.

3.4.2 Pipeline CI/CD (GitHub Actions)

Le pipeline (.github/workflows/deploy.yml) est déclenché à chaque *push* sur main/master ou manuellement via workflow_dispatch (avec choix de l’action : deploy ou destroy).

Table 3.7. Jobs du pipeline GitHub Actions

Job	Actions réalisées
bootstrap	Crée le bucket S3, initialise Terraform (backend S3), importe les ressources AWS existantes.
detect-iam-role	Détecte un rôle IAM compatible parmi LabRole, EMR_EC2_DefaultRole, AmazonEKSNodeRole.
detect-changes	Détecte si les fichiers terraform/** ont été modifiés.
sonar	Build Maven + analyse statique SonarCloud du backend.
terraform	terraform init, plan et apply si nécessaire. Exporte les URLs ECR et le nom du cluster.
build	Build et push des images Docker (tag GITHUB_SHA::8 + latest) vers ECR backend et frontend.
deploy	aws eks update-kubeconfig, création du namespace et du secret skilltree-secrets, injection des images ECR dans les manifests via sed, kubectl apply des cinq manifests, vérification du rollout, affichage de l’URL ALB.
destroy	(Manuel) Suppression du NodeGroup puis terraform destroy.

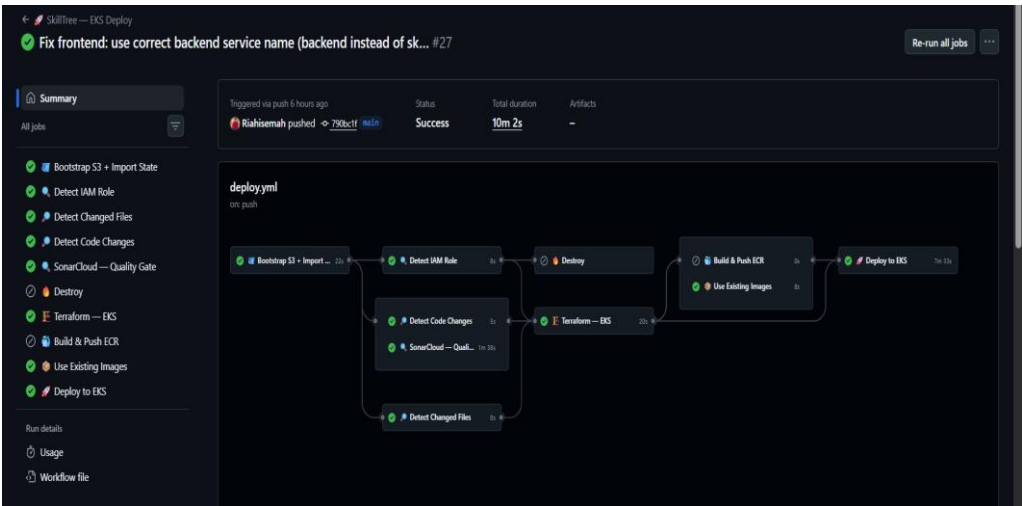


Figure 3.8. Pipeline CI/CD GitHub Actions (déploiement AWS EKS)

3.4.3 Manifests Kubernetes (EKS)

Les manifests applicatifs sont regroupés dans le dossier k8s-manifests/ et appliqués dans l’ordre par le pipeline.

Table 3.8. Manifests Kubernetes et leur rôle

Manifest	Contenu
01-namespace-configmap.yaml	Namespace skilltree + ConfigMap skilltree-config (URL DB, profil Spring, région AWS...).
02-mariadb.yaml	Service headless mariadb + StatefulSet MariaDB 10.11 (volume emptyDir).
03-backend.yaml	Service ClusterIP backend + Deployment Spring Boot. initContainer attendant MariaDB. Image injectée par le pipeline (IMAGE_PLACEHOLDER_BACKEND).
04-frontend.yaml	Service ClusterIP frontend + Deployment Nginx/React. initContainer attendant le backend. Image injectée par le pipeline (IMAGE_PLACEHOLDER_FRONTEND).
05-ingress.yaml	Ingress skilltree-ingress (classe alb, internet-facing, target-type IP). Routage : /api backend, /actuator backend, / frontend.

3.5 Monitoring et Observabilité

Cette phase vise à assurer la supervision continue de l'application FormationPro déployée sur AWS EKS. Afin de garantir la disponibilité des services, la visibilité sur les performances et la détection proactive des incidents, plusieurs mécanismes de monitoring et d'alerting ont été mis en place à l'aide de Kubernetes et des services AWS CloudWatch.

3.5.1 Healthchecks Kubernetes

Des probes de type Startup, Liveness et Readiness ont été configurées pour les services frontend et backend. Ces vérifications reposent notamment sur les endpoints / et /actuator/health.

Les startups probes permettent de vérifier que l'application démarre correctement avant de recevoir du trafic. Les liveness probes détectent les applications bloquées et déclenchent automatiquement un redémarrage du pod concerné. Enfin, les readiness probes garantissent qu'un pod ne reçoit du trafic que lorsqu'il est totalement opérationnel.

Grâce à ces mécanismes, Kubernetes assure automatiquement le maintien de la disponibilité des services déployés.

3.5.2 Dashboard CloudWatch

Afin de centraliser les informations de supervision, un tableau de bord CloudWatch nommé **SkillTree-Dashboard-Avance** a été créé.

Ce dashboard regroupe les principales métriques du cluster Kubernetes et de l'application, notamment :

- L'état général du cluster EKS ;
- L'utilisation du processeur et de la mémoire des pods ;
- Le trafic réseau entrant et sortant ;
- Les métriques du serveur API Kubernetes ;
- Les performances du stockage etcd ;
- Les indicateurs liés aux webhooks et aux composants du cluster.

Cette centralisation permet d'obtenir une vision en temps réel de l'état de santé de l'infrastructure et facilite l'analyse des performances.

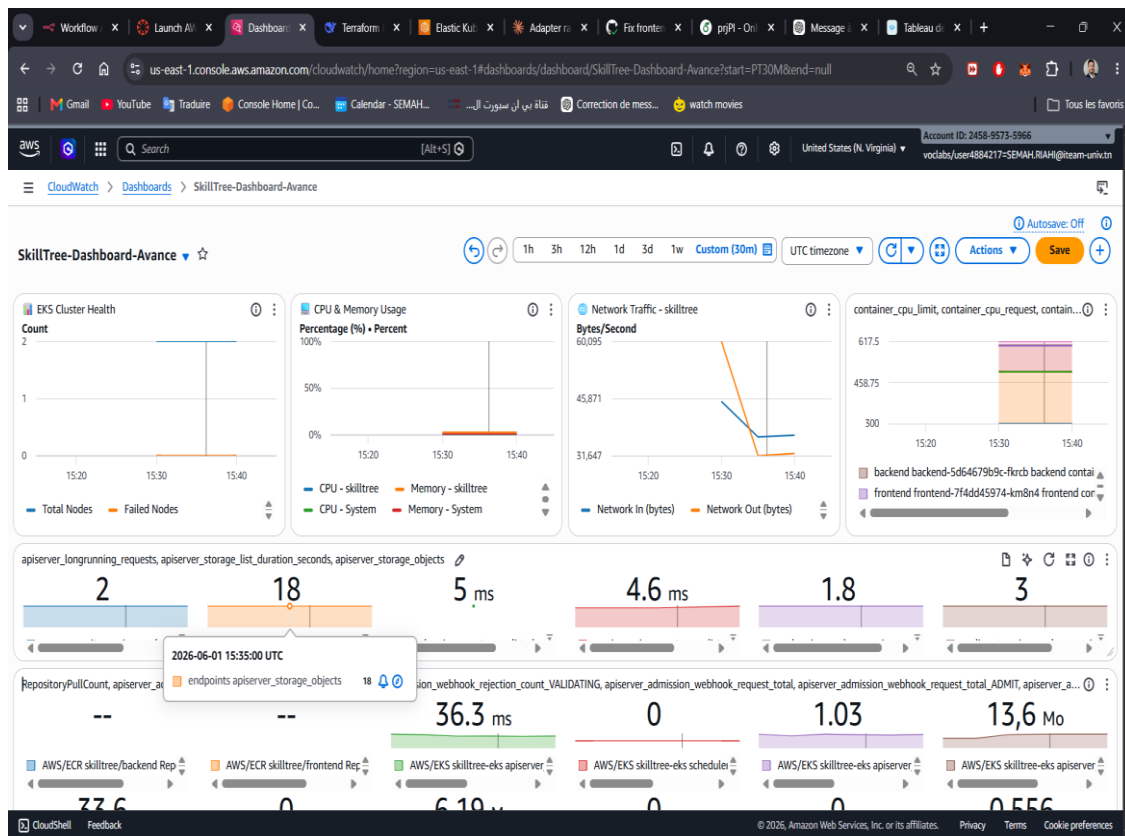


Figure 3.9. Dashboard CloudWatch SkillTree-Dashboard-Avance.

3.5.3 Alarmes CloudWatch

Afin de détecter rapidement les anomalies, des alarmes CloudWatch ont été configurées sur les métriques critiques des pods de l'application.

Les principales alarmes mises en place sont présentées dans le tableau suivant :

Tableau 3.9 : Seuils de surveillance des ressources système

Alarme	Métrique surveillée	Seuil
skilltree-high-cpu	pod_cpu_utilization	> 80 %
skilltree-high-memory	pod_memory_utilization	> 85 %

Lorsque l'un des seuils configurés est dépassé pendant plusieurs périodes consécutives, CloudWatch déclenche automatiquement une notification via Amazon SNS afin d'informer les administrateurs du système.

Cette approche permet une détection proactive des incidents et contribue à améliorer la disponibilité globale de la plateforme.

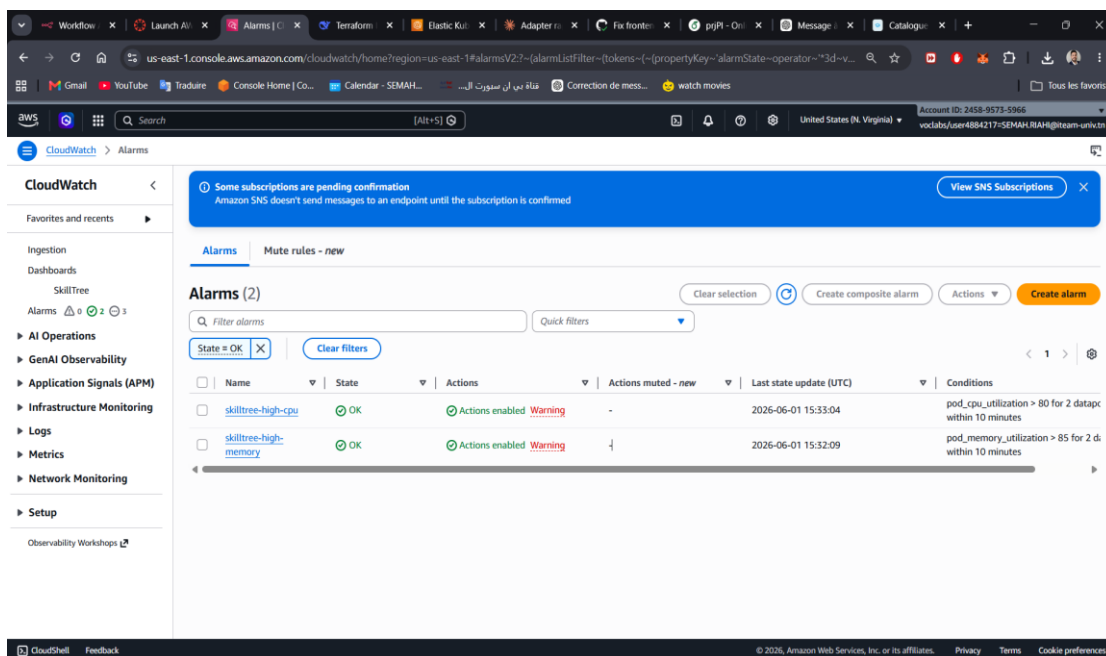


Figure 3.8. État des alarmes CloudWatch.

Ainsi, l'intégration des mécanismes de monitoring, de supervision et d'alerting renforce la fiabilité de la plateforme FormationPro et facilite l'exploitation quotidienne de l'infrastructure déployée sur AWS.

Conclusion du chapitre

Ce chapitre a présenté la réalisation concrète de FormationPro. L'implémentation couvre l'authentification JWT, le catalogue de formations, les inscriptions, le référentiel de compétences, les objectifs manager, les feedbacks, les notifications et les rapports analytiques. La stratégie de tests (fonctionnels, sécurité, intégration) a validé l'ensemble des parcours avec un taux de succès de 100%. Enfin, le déploiement sur Amazon EKS est entièrement automatisé par Terraform et GitHub Actions, assurant une livraison continue et reproductible. L'ensemble des exigences fonctionnelles et non fonctionnelles a ainsi été respecté.

Conclusion Générale

Résumé des réalisations

Le projet **FormationPro (Skill Tree)** a abouti à la livraison d'une plateforme web complète de gestion des formations professionnelles, développée selon la méthodologie Agile Scrum en quatre sprints successifs. L'application offre :

- Un système d'authentification sécurisé (JWT, BCrypt, RBAC) avec trois rôles distincts.
- Un catalogue de formations administrable par les RH, consultable par tous.
- Un parcours employé complet : inscription, suivi de progression, feedback.
- Des outils de pilotage manager : vue équipe, objectifs formalisés avec deadline.
- Une couche d'évaluation d'impact post-formation.
- Un module de notifications persistées.
- Des rapports analytiques pour la prise de décision RH/manager.
- Un déploiement cloud sur **Amazon EKS** avec infrastructure as code Terraform, pipeline CI/CD GitHub Actions et analyse qualité SonarCloud.

Valeur ajoutée du projet

FormationPro répond directement à la problématique de fragmentation des outils de gestion des compétences en proposant une solution unifiée, multi-rôles et déployable automatiquement sur le cloud AWS. La plateforme transforme des processus manuels et dispersés en un système centralisé, traçable et piloté par la donnée. L'intégration d'une infrastructure as code Terraform et d'un pipeline CI/CD multi-jobs entièrement automatisé du build de l'image Docker au déploiement sur EKS apporte un niveau de maturité DevOps cloud rarement atteint dans un contexte de projet de fin d'études.

Perspectives d'amélioration

FormationPro constitue une base solide, extensible selon les axes suivants :

- **Persistence MariaDB** : remplacement de l'emptyDir par un PVC EBS pour une persistance des données entre les redéploiements.
- **HTTPS/TLS sur ALB** : activation du listener HTTPS avec certificat ACM sur l'ALB.
- **Monitoring cloud** : intégration d'Amazon CloudWatch Container Insights ou déploiement d'une stack Prometheus/Grafana sur EKS.
- **Pagination et filtrage avancés** : formations, utilisateurs, rapports.
- **Notifications temps réel** : remplacement des notifications persistées par Web-

Socket (STOMP).

- **Export PDF/Excel** : génération de rapports téléchargeables.
- **Intégration SSO** : authentification via OAuth2/Cognito ou Keycloak.
- **Tests automatisés** : couverture unitaire (JUnit, Mockito) et tests end-to-end (Cypress, Playwright).
- **Environnements multiples** : gestion de workspaces Terraform (dev/staging/-prod).
- **Intelligence artificielle** : recommandation de formations basée sur le profil de compétences.

Bibliographie

- [1] Walls C. (2022). *Spring Boot in Action*. 2^{ème} édition. Manning Publications.
- [2] Spilca L. (2020). *Spring Security in Action*. Manning Publications.
- [3] Wieruch R. (2022). *The Road to React*. Leanpub.
- [4] Luksa M. (2023). *Kubernetes in Action*. 2^{ème} édition. Manning Publications.
- [5] Jones M. et al. (2015). *JSON Web Token (JWT)*. RFC 7519. IETF. <https://datatracker.ietf.org/doc/html/rfc7519>
- [6] Schwaber K., Sutherland J. (2020). *The Scrum Guide*. Scrum.org. <https://scrumguides.org>
- [7] Kane M., Matthias K. (2023). *Docker : Up & Running*. 3^{ème} édition. O'Reilly Media.
- [8] Brikman Y. (2022). *Terraform : Up & Running*. 3^{ème} édition. O'Reilly Media.
- [9] Documentation officielle Amazon EKS. <https://docs.aws.amazon.com/eks/>
- [10] Documentation officielle GitHub Actions. <https://docs.github.com/en/actions>
- [11] Documentation SonarCloud. <https://docs.sonarcloud.io>